

IEORE4004

Optimization Models and Methods

Yaren Bilge Kaya, PhD

Columbia University
Department of Industrial Engineering and Operations Research



Common IP Problems: The Traveling Salesperson Problem

- IP Formulation and the Problem of Subtour Growth
- Miller, Tucker, Zemlin Formulation

Traveling Salesperson Problem

Perhaps the most classical of optimization problems

- A salesperson needs to design a route ("tour") through n cities, returning to their original location
 - Would like to do so in the shortest possible distance

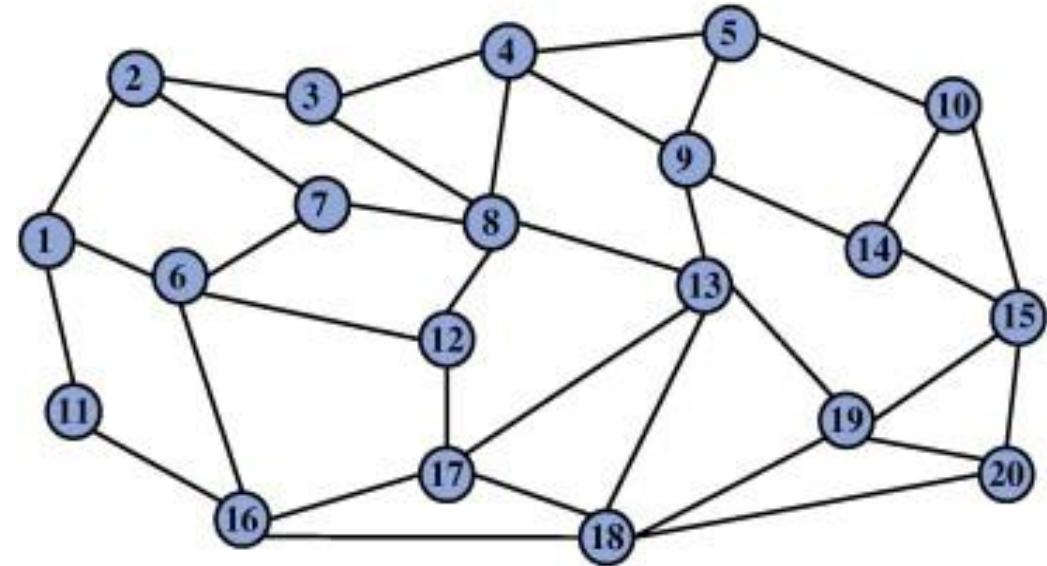


Figure. Structure of the emergency logistics network. Adapted from “Route selection for Emergency Logistics Management: A bio-inspired algorithm,” by X. Zhang, Z. Zhang, Y. Zhang, D. Wei, and Y. Deng, 2013, *Safety Science*, 54, p. 89 (<https://doi.org/10.1016/j.ssci.2012.12.003>). Copyright 2013 by Elsevier.

Traveling Salesperson Problem

- Has a wide variety of applications:
 - Routing and Logistics

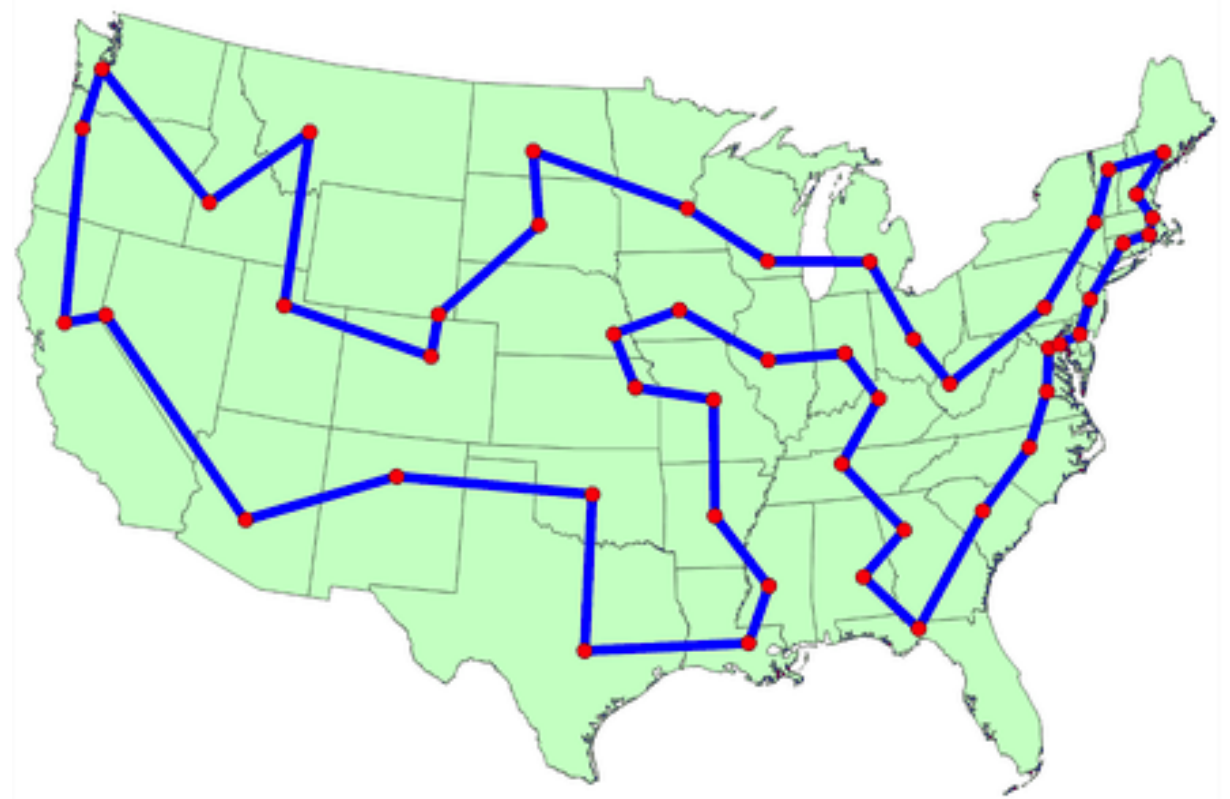


Figure. Traveling salesman problem - Cornell University Computational Optimization Open Textbook - Optimization Wiki

Traveling Salesperson Problem

- Has a wide variety of applications:
 - Routing and Logistics

Let's try solving the TSP together!

Navigate here:

https://algorithms.discrete.ma.tum.de/graph-games/tsp-game/index_en.html

Choose **Italien (Italy)** as your map

Number of cities: **30**

Use **Euclidean** distance

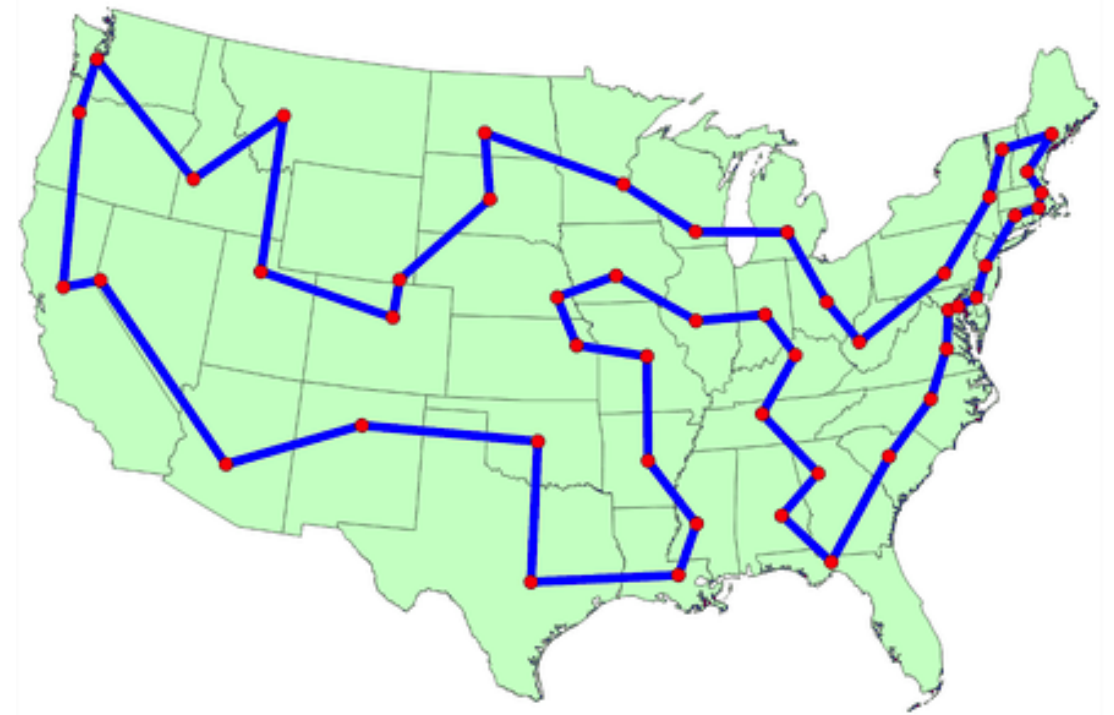


Figure. Traveling salesman problem - Cornell University Computational Optimization Open Textbook - Optimization Wiki

Traveling Salesperson Problem

- Wide variety of applications:
 - Routing and Logistics
 - Integrated circuits design

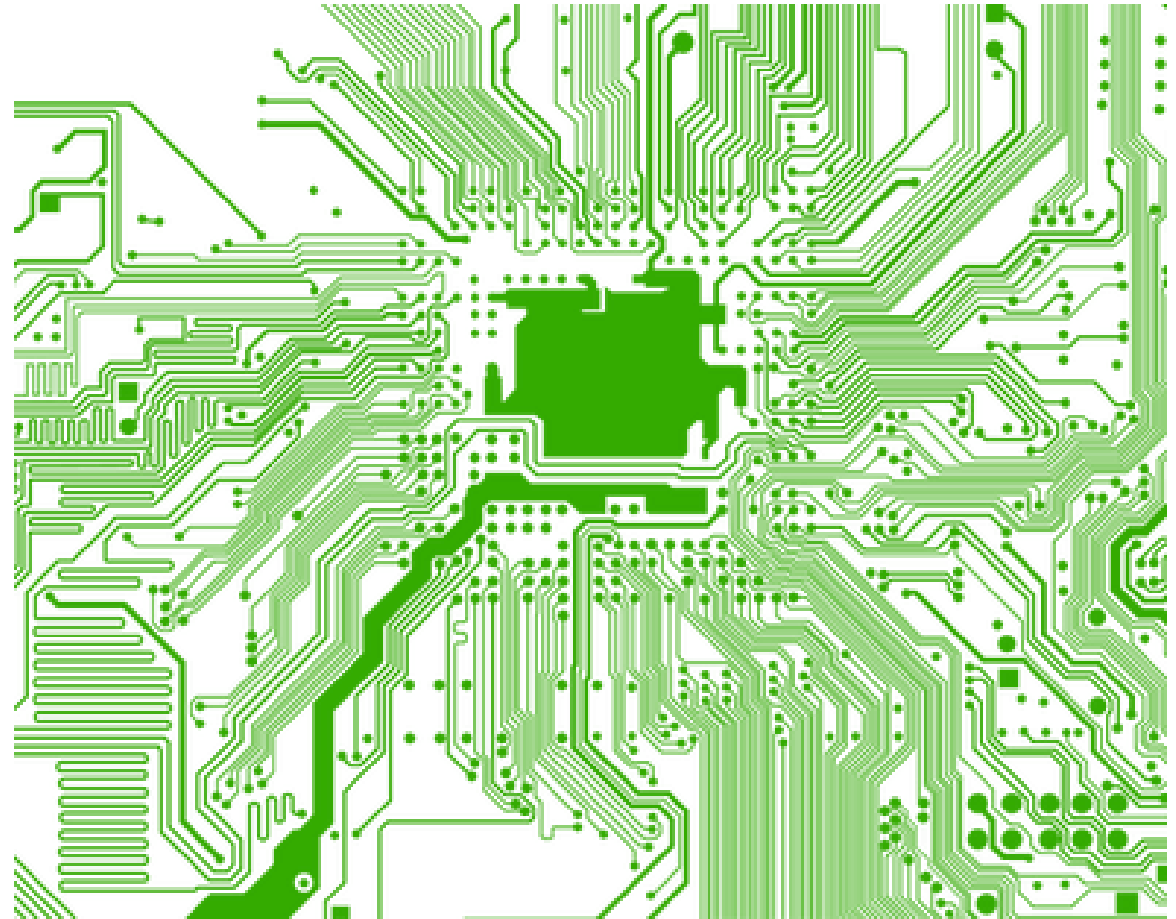
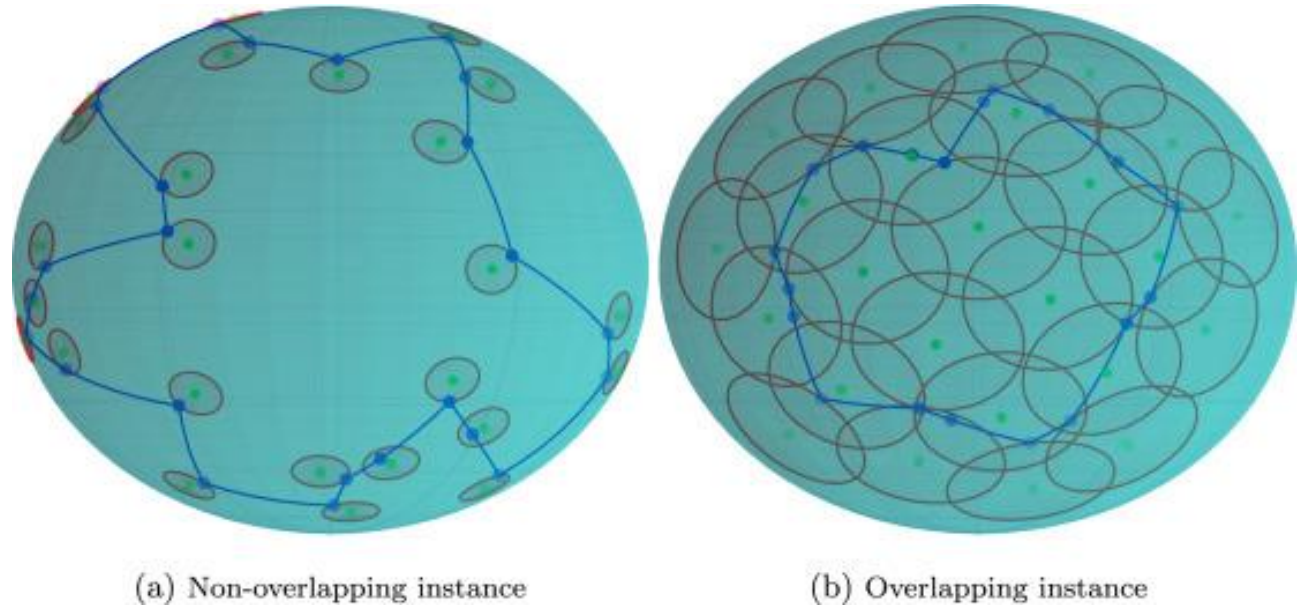


Figure. Top Online Circuit Design Courses for Beginners by Mukta Manshi,
<https://www.skyfilabs.com/blog/top-online-circuit-design-courses-for-beginners>

Traveling Salesperson Problem

- Wide variety of applications:
 - Routing and Logistics
 - Integrated circuits design
 - Imaging of celestial* objects



**astronomical*

Figure. Traveling Salesman Problem with neighborhoods on a sphere in reflectance transformation imaging scenarios. Jindřiška Deckerová, [Jan Faigl](#), and Vít Krátký.
<https://doi.org/10.1016/j.eswa.2022.116814>

Traveling Salesperson Problem

- Wide variety of applications:
 - Routing and Logistics
 - Integrated circuits design
 - Imaging of celestial* objects
 - Artwork!

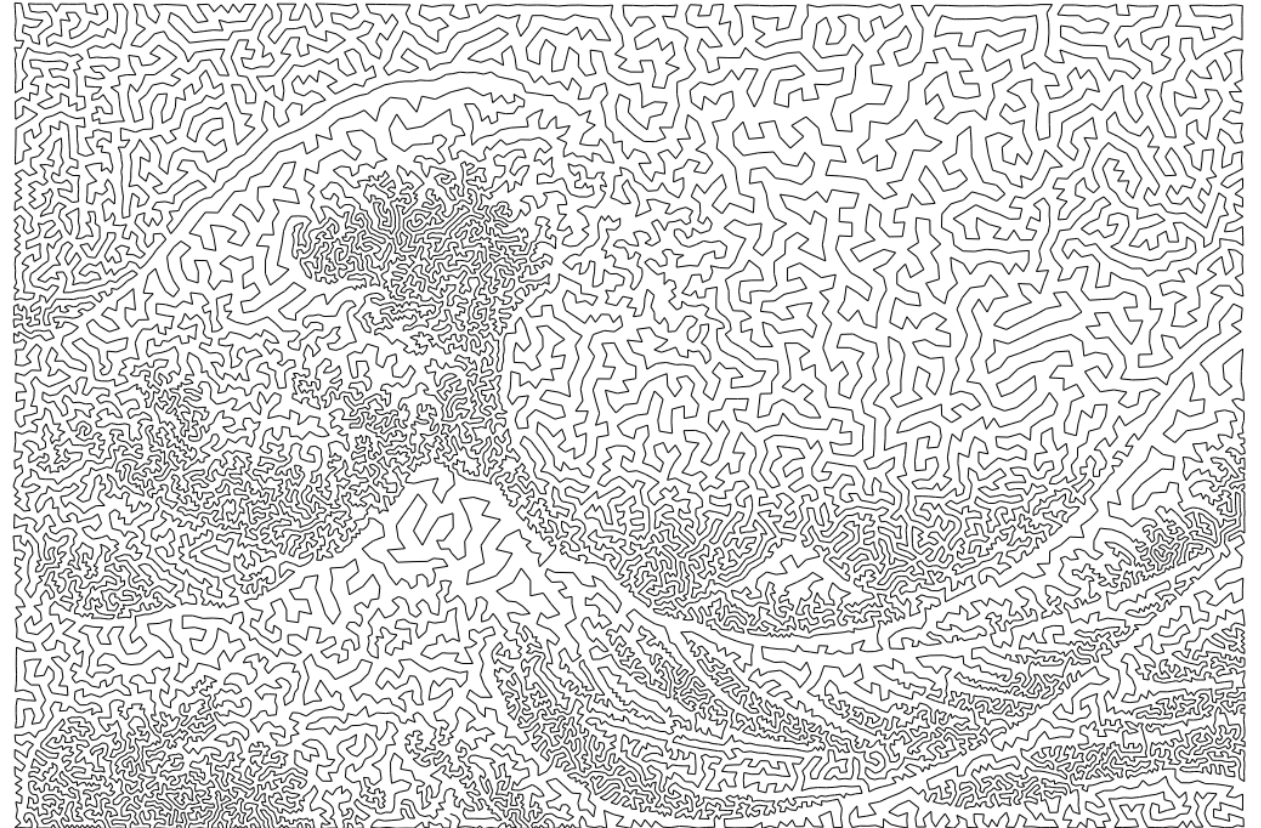
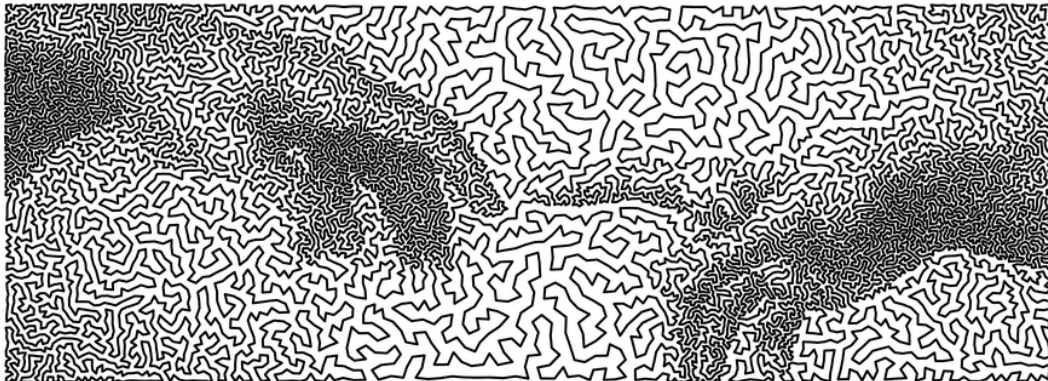
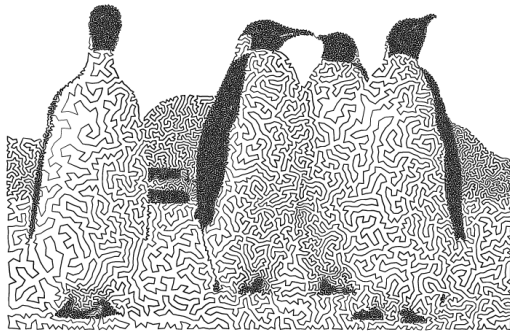
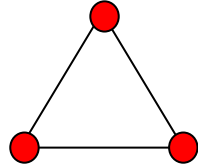


Figure. Optimization Art by Bob Bosch. <https://www2.oberlin.edu/math/faculty/bosch/tspart-page.html>

How Hard Could That Be?

What if there just three cities ... how many unique ways to find a circuit through them?

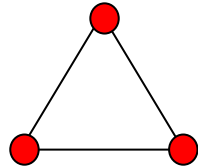
- Just one way!



How Hard Could That Be?

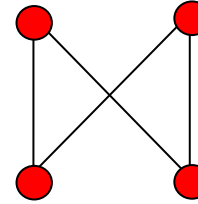
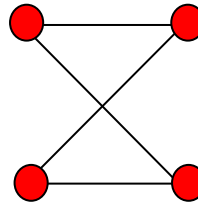
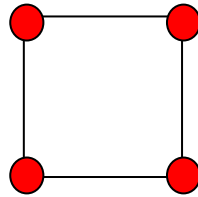
What if there just three cities ... how many unique ways to find a circuit through them?

- Just one way!



What about four cities?

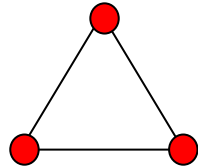
... There are 3 ways



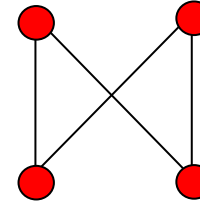
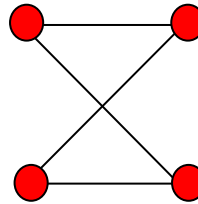
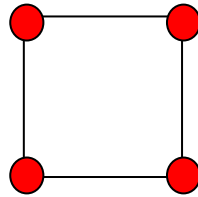
How Hard Could That Be?

What if there just three cities ... how many unique ways to find a circuit through them?

- Just one way!



What about four cities?
... There are 3 ways



And five cities? ...Twelve ways

Six cities? Eight? Ten? Twelve? Twenty?

60; 20,160; 1,814,400; 239,500,800; 1,216,451,004,088,320,000 ways

From A Procter & Gamble Ad - 1962:

Imagine that Toody and Muldoon want to drive around the country and visit each of the 33 locations represented by dots on the contest map, and that in doing so, they want to travel the shortest possible route. You should plan a route for them from location to location which will result in the shortest total mileage from Chicago, Illinois back to Chicago, Illinois.

10,000 \$ in 1962 correspond to \$80,000 in today's value.



Source: University of Houston – *The Engines of Our Ingenuity*

Other Facts About The TSP

- This problem has been studied very well
- It is NP-hard (Non-deterministic Polynomial-time hard) *difficult to solve, computationally speaking*
- That said, large instances have been solved optimally (over 30,000 "cities" – actually, a microchip layout problem)
- Very large instances have been solved approximately (ten million cities to within 1 or so percent of optimum)

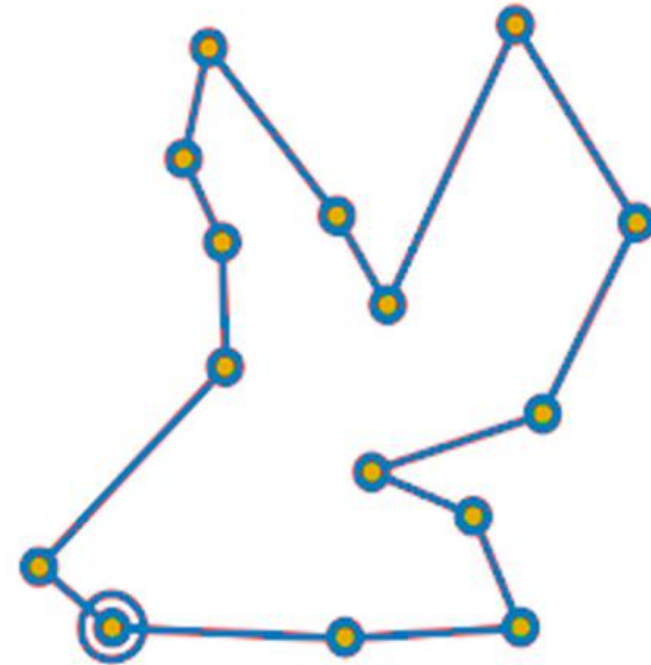
How To Solve TSP Computationally?

While the basic TSP resembles a totally connected network, unfortunately it does not share the beneficial integrality properties of other network flow problems

- Heuristic approaches exist:
 - Find good solutions
 - No guarantee of quality

Cities: 15
Length: 1,251.00

You are within 1.0501% of the optimum.

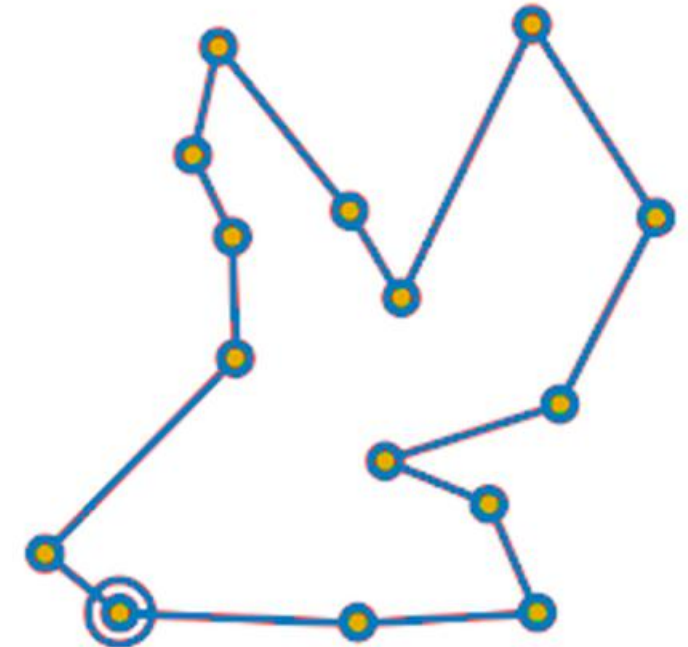


How To Solve TSP Computationally?

While the basic TSP resembles a totally connected network, unfortunately it does not share the beneficial integrality properties of other network flow problems

- Heuristic approaches exist:
 - Find good solutions
 - No guarantee of quality
- We will approach it as a binary integer program
 - Guarantee on global optimality
 - Multiple ways to do this
 - We consider a *symmetric* TSP formulation, where the distance from i to j , is the same from j to i

You are within 1.0501% of the optimum.



Cities: 15
Length: 1,251.00

Formulating The TSP (Symmetric)

This can be done completely with binary variables!

- Suppose N is a set of cities, indexed by i and j .
- Let d_{ij} be the distance from city i to city j , $i \neq j$.
- Let $x_{ij} = \begin{cases} 1 & \text{if arc from node } i \text{ to node } j \text{ is traversed, } i \neq j; \\ 0 & \text{otherwise.} \end{cases}$

Formulating The TSP (Symmetric)

This can be done completely with binary variables!

- Suppose N is a set of cities, indexed by i and j .
- Let d_{ij} be the distance from city i to city j , $i \neq j$.
- Let $x_{ij} = \begin{cases} 1 & \text{if arc from node } i \text{ to node } j \text{ is traversed, } i \neq j; \\ 0 & \text{otherwise.} \end{cases}$

$$\begin{array}{ll} \min & \sum_{i \in N} \sum_{j \in N: j \neq i} d_{ij} x_{ij} \\ \text{subject to} & \sum_{i \in N: i \neq j} x_{ij} = 1 \quad \forall j \in N, \\ & \sum_{j \in N: j \neq i} x_{ij} = 1 \quad \forall i \in N, \\ & x_{ij} \in \{0, 1\} \quad \forall i \in N, \quad \forall j \in N. \end{array}$$

Formulating The TSP (Symmetric)

This can be done completely with binary variables!

- Suppose N is a set of cities, indexed by i and j .
- Let d_{ij} be the distance from city i to city j , $i \neq j$.
- Let $x_{ij} = \begin{cases} 1 & \text{if arc from node } i \text{ to node } j \text{ is traversed, } i \neq j; \\ 0 & \text{otherwise.} \end{cases}$

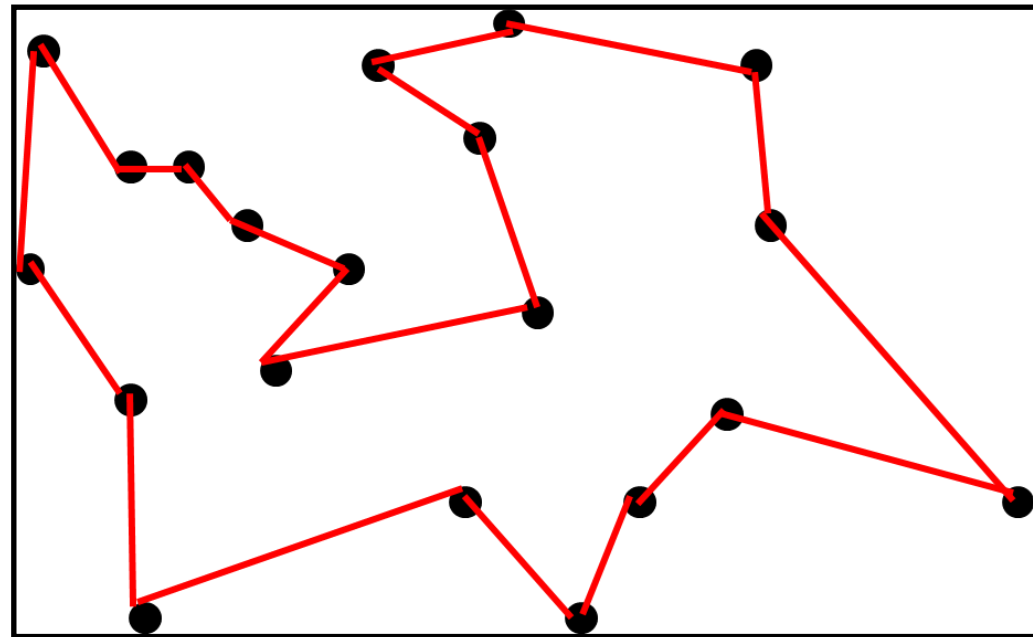
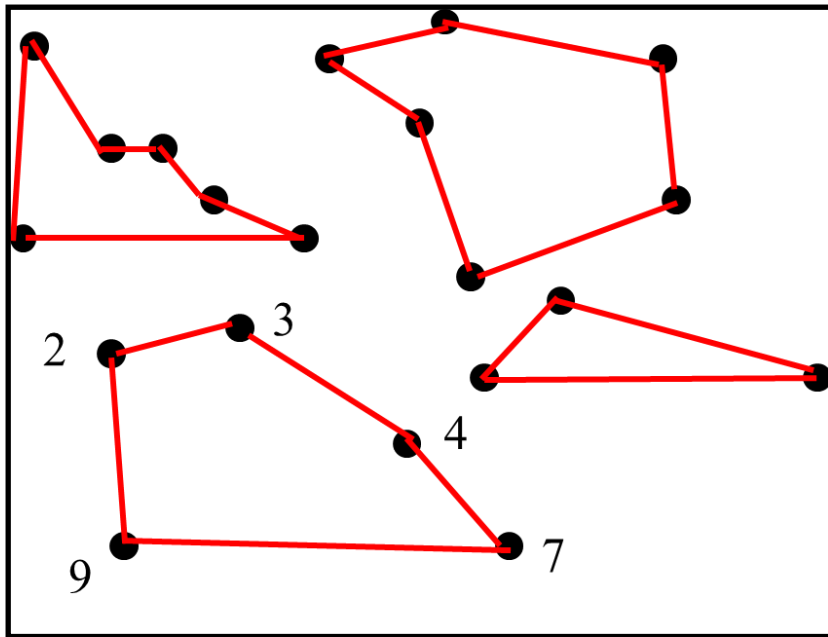
$$\begin{array}{ll} \min & \sum_{i \in N} \sum_{j \in N: j \neq i} d_{ij} x_{ij} \\ \text{subject to} & \sum_{i \in N: i \neq j} x_{ij} = 1 \quad \forall j \in N, \\ & \sum_{j \in N: j \neq i} x_{ij} = 1 \quad \forall i \in N, \\ & x_{ij} \in \{0, 1\} \quad \forall i \in N, \quad \forall j \in N. \end{array}$$

Well, almost ...
something is still
missing ...
Anyone see it?

Subtours

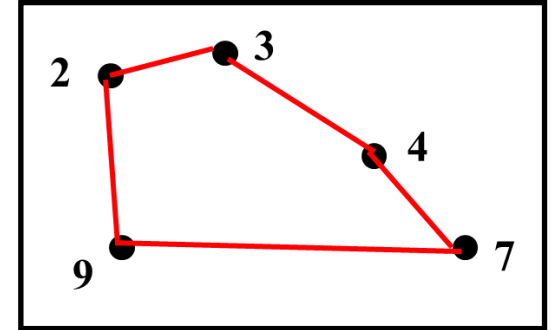
Subtours!

- The previous formulation may permit *subtours*:
Smaller tours that connect a strict subset of cities



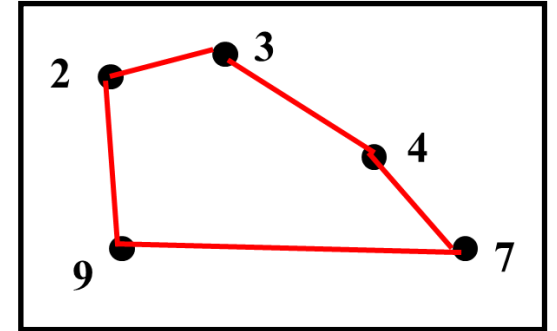
A Subtour-Breaking Constraint

- Let S be any *proper* subset of nodes, $|S| \geq 3$
 - For example, $S = \{2, 3, 4, 7, 9\}$
- Two key observations:
 - A subtour that includes all nodes of S has $|S|$ arcs
 - A tour for the entire (sub)network of S *cannot contain more than* $|S| - 1$ arcs having two endpoints in S



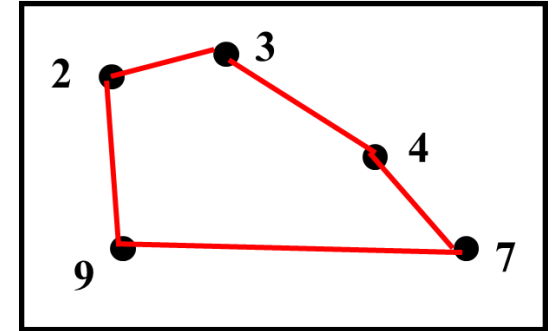
A Subtour-Breaking Constraint

- Let S be any *proper* subset of nodes, $|S| \geq 3$
 - For example, $S = \{2, 3, 4, 7, 9\}$
- Two key observations:
 - A subtour that includes all nodes of S has $|S|$ arcs
 - A tour for the entire (sub)network of S *cannot contain more than* $|S| - 1$ arcs having two endpoints in S
- To forbid a subtour on subset S of nodes, add constraint:
$$\sum_{i \in S} \sum_{j \in S; j \neq i} x_{ij} \leq |S| - 1$$
 - This constraint sums **all** arcs in the subset S (i.e., not just the active ones)



A Subtour-Breaking Constraint

- Let S be any *proper* subset of nodes, $|S| \geq 3$
 - For example, $S = \{2, 3, 4, 7, 9\}$
- Two key observations:
 - A subtour that includes all nodes of S has $|S|$ arcs
 - A tour for the entire (sub)network of S *cannot contain more than* $|S| - 1$ arcs having two endpoints in S
- To forbid a subtour on subset S of nodes, add constraint:
$$\sum_{i \in S} \sum_{j \in S; j \neq i} x_{ij} \leq |S| - 1$$
 - This constraint sums **all** arcs in the subset S (i.e., not just the active ones)



Easy enough, right? How many possible subtours exist?

Subtour Growth!

- Adding all subtour constraints is not a problem for small networks, say, with 6 to 10 nodes (cities)
 - However, the number of subtours grows exponentially with the number of cities
 - Prohibitive to add all of them
 - Any ideas on what to do?

Subtour Growth!

- Adding all subtour constraints is not a problem for small networks, say, with 6 to 10 nodes (cities)
 - However, the number of subtours grows exponentially with the number of cities
 - Prohibitive to add all of them
 - Any ideas on what to do?

1. Include only some of the constraints, and solve
2. If a subtour appears as part of the solution, add a new subtour elimination constraint and solve again

Miller-Tucker-Zemlin (MTZ) Formulation

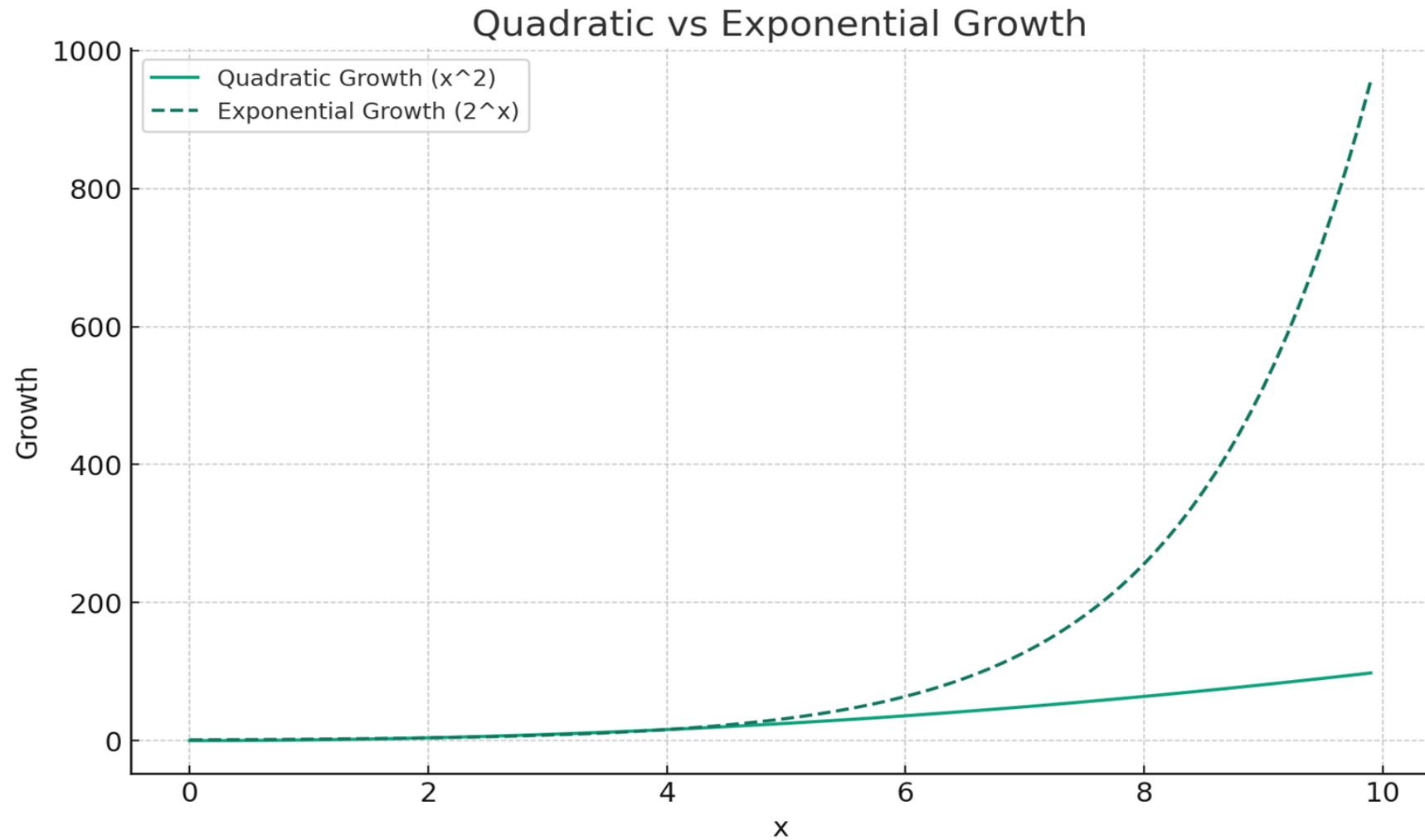
An Alternative...

There is an alternative to adding (exponentially many) subtour elimination constraints

Miller-Tucker-Zemlin (MTZ) subtour-elimination constraints

- They ensure that each node carries a value called the “potential”, u_j , that conveys a sense of distance from the origin
- The node potential indicates the **order of the corresponding node in tour**

**The number of MTZ constraints is quadratic in the size of the node set
(as compared to an exponential number!)**



An Alternative...

There is an alternative to adding (exponentially many) subtour elimination constraints

Miller-Tucker-Zemlin (MTZ) subtour-elimination constraints

- They ensure that each node carries a value called the “potential”, u_j , that conveys a sense of distance from the origin
- The node potential indicates the **order of the corresponding node in tour**
- The number of MTZ constraints is quadratic in the size of the node set (as compared to an exponential number!)
- However, the MTZ constraints are in general *weaker*

than
$$\sum_{i \in S} \sum_{j \in S; j \neq i} x_{ij} \leq |S| - 1$$

Miller-Tucker-Zemlin TSP Formulation

$$\begin{array}{ll}\min & \sum_{i \in N} \sum_{j \in N} d_{ij} x_{ij} \\ \text{subject to} & \sum_{j \in N \setminus \{i\}} x_{ij} = 1, \quad \forall i \in N \\ & \sum_{j \in N \setminus \{i\}} x_{ji} = 1, \quad \forall i \in N\end{array}$$

$$u_i + 1 \leq u_j + (n - 1)(1 - x_{ij}), \quad \forall i \in N, j \in N, i \neq j \neq 1$$

$$x_{ii} = 0, \quad \forall i \in N$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in N, j \in N, i \neq j$$

$$u_1 = 1,$$

$$2 \leq u_i \leq n, \quad i \neq 1.$$

Usually, the starting city (say city 1) is fixed with $u_1 = 1$ to anchor the tour and provide a reference point for the ordering.

Open Form of MTZ for TSP - Example

Problem Setup:

We have 4 cities: A, B, C, D (labeled 1 through 4). Define binary variable :

$$x_{ij} = \begin{cases} 1 & \text{if the tour goes from city } i \text{ to city } j \\ 0 & \text{otherwise} \end{cases}$$

- u_i = position of city i in the tour (only for $i = 2$ to 4 ; city 1 is the depot with $u_1 = 1$)

Open Form of MTZ for TSP - Example

Objective Function:

Minimize total travel cost:

$$\min \sum_{i=1}^4 \sum_{j=1}^4 c_{ij} \cdot x_{ij}$$

Subject to:

- Visit every city exactly once

$$\sum_{j=1, j \neq i}^4 x_{ij} = 1 \quad \forall i = 1, \dots, 4$$

$$\sum_{i=1, i \neq j}^4 x_{ij} = 1 \quad \forall j = 1, \dots, 4$$

Open Form of MTZ for TSP - Example

- MTZ Subtour Elimination Constraints for $i \neq j, i, j \in \{2, 3, 4\}$:

$$u_i + 1 \leq u_j + (3)(1 - x_{i,j})$$

- Bounds on u variables:

$$2 \leq u_i \leq 4 \quad \forall i = 2, 3, 4$$

- Binary and Continuous variables:

$$x_{ij} \in \{0, 1\}, \quad u_i \in \mathbb{Z}$$

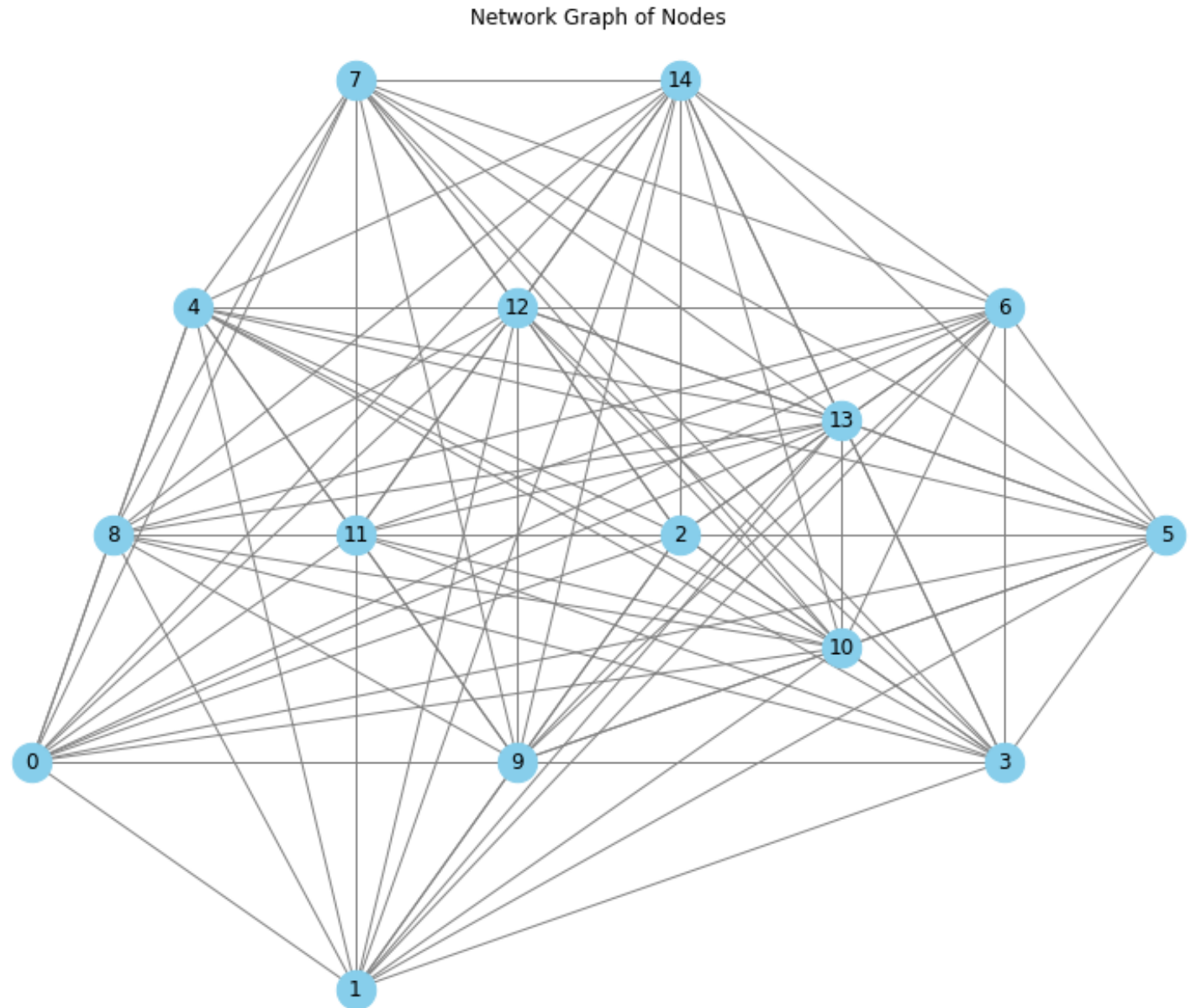
Open Form of MTZ for TSP - Example

Advantages of using it:

- These MTZ constraints prevent subtours by ensuring a consistent order in which cities are visited.
- The node potential u_j gives a position in the tour, so by this way, the ordering breaks if a tour forms a cycle without visiting all cities.

Implementing TSP with MTZ

Let's try to solve the TSP for the given network using Gurobi!



Different Approaches to Solve the TSP

- The brute-force approach

Also known as the naive approach, calculates and compares all possible permutations of routes or paths to determine the shortest unique solution

- The nearest neighbor method

Randomly selects a starting node, then finds the closest unvisited node and add it to the sequencing.

Then, moves to the next node and repeat the process of finding the nearest unvisited node until all nodes are included in the tour.

Finally, we return to the starting city to complete the cycle.

Different Approaches to Solve the TSP

- The brute-force search approach

Also known as the naive approach, calculates and compares all possible permutations of routes or paths to determine the shortest unique solution (becomes impractical even for only 20 cities!)

- The nearest neighbor method (Heuristics)

Randomly selects a starting node, then finds the closest unvisited node and add it to the sequencing.

Then, moves to the next node and repeat the process of finding the nearest unvisited node until all nodes are included in the tour.

Finally, we return to the starting city to complete the cycle.

Different Approaches to Solve the TSP

- Concorde TSP Solver (Exact Solver)

“is widely regarded as the fastest TSP solver, for large instances, currently in existence”

While our MTZ and LP formulations use integer programming to find optimal solutions through branch-and-bound, Concorde implements sophisticated cutting-plane algorithms and heuristics optimized for TSP structure.

Link: <https://www.math.uwaterloo.ca/tsp/concorde/index.html>